

ReplicaSet e ReplicationController

Obiettivi di Apprendimento

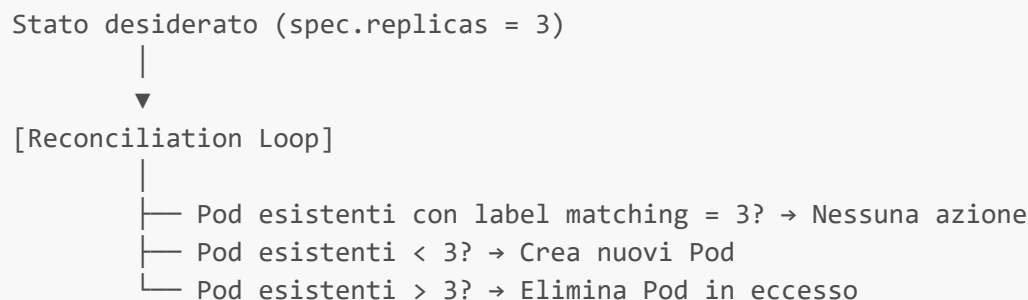
Al termine di questa sezione il corsista sarà in grado di:

- Spiegare il ruolo di **ReplicaSet** come controller che garantisce un numero preciso di Pod in esecuzione
- Distinguere **ReplicaSet** da **ReplicationController** e capire perché quest'ultimo è deprecato
- Interpretare la struttura YAML di un ReplicaSet, in particolare i selettori **matchLabels** e **matchExpressions**
- Spiegare il meccanismo di riconciliazione e come il ReplicaSet "adotta" Pod esistenti
- Scalare un ReplicaSet manualmente e capire l'impatto dell'eliminazione manuale di un Pod
- Capire perché in produzione si usa sempre un **Deployment** sopra a un **ReplicaSet**

Teoria e Concetti Chiave

Il Problema che Risolve un ReplicaSet

Un singolo Pod non è resiliente: se il nodo che lo ospita va down, il Pod scompare. Il **ReplicaSet** (RS) risolve questo problema garantendo che in qualsiasi momento sia sempre attivo un numero preciso (**replicas**) di Pod con una determinata etichetta (label). Se un Pod viene eliminato o crasha definitivamente, il RS ne crea uno nuovo in sostituzione.



Struttura di un ReplicaSet

Un ReplicaSet ha tre sezioni principali in **spec**:

Campo	Descrizione
replicas	Numero desiderato di Pod
selector	Selettore per identificare i Pod che il RS gestisce
template	Template del Pod da creare (identico a un PodSpec)

⚠ Attenzione: Il **selector** del ReplicaSet e i **labels** nel **template.metadata.labels** devono **corrispondere**. Se non corrispondono, il manifest verrà rifiutato dall'API server.

Selettori: matchLabels vs matchExpressions

Il **selector** di un ReplicaSet supporta due forme:

matchLabels — forma semplice, ogni coppia chiave/valore deve essere presente:

```
selector:
  matchLabels:
    app: webapp
    tier: frontend
```

Equivale a: *seleziona i Pod che hanno ENTRAMBE le label **app=webapp** E **tier=frontend***

matchExpressions — forma avanzata per condizioni più complesse:

```
selector:
  matchExpressions:
    - key: app
      operator: In
      values: ["webapp", "webapp-v2"]      # app IN (webapp, webapp-v2)
    - key: tier
      operator: NotIn
      values: ["database"]                  # tier NOT IN (database)
    - key: environment
      operator: Exists                      # il label 'environment' deve esistere
```

Operator	Significato
In	Il valore della label deve essere in una lista specificata
NotIn	Il valore della label NON deve essere in una lista specificata
Exists	La label deve esistere (qualsiasi valore)
DoesNotExist	La label non deve esistere

Adozione di Pod Esistenti

Il meccanismo di "adozione" è un aspetto critico del ReplicaSet: il RS non crea necessariamente nuovi Pod, ma cerca Pod esistenti che corrispondano al selettore. Se trova Pod liberi (senza **ownerReference**), li "adotta". Questo può avere effetti indesiderati se si creano Pod manualmente con le stesse label di un RS attivo.

⚠ Attenzione: Creare Pod manualmente con label che corrispondono a un ReplicaSet esistente può causare l'eliminazione immediata del Pod appena creato (se il RS ha già raggiunto il numero desiderato) o può portare a un "sorpasso" del numero desiderato con successiva eliminazione dei Pod in eccesso.

ReplicaSet vs ReplicationController

ReplicationController (RC) è il predecessore del **ReplicaSet**, introdotto nelle prime versioni di Kubernetes. Le differenze principali:

Caratteristica	ReplicaSet	ReplicationController
Selettore	<code>selector.matchLabels</code> + <code>matchExpressions</code>	Solo uguaglianza semplice (<code>selector flat</code>)
Status	Corrente (supportato attivamente)	Deprecato (ancora funzionante ma non consigliato)
Usato da	Deployment	DeploymentConfig (OCP legacy)
Espressività	Alta (operatori In, NotIn, Exists)	Bassa (solo = e !=)

💡 **Suggerimento:** Se si incontra un **ReplicationController** in un cluster OCP esistente, è probabilmente legacy creato da un **DeploymentConfig**. In nuovi progetti usare sempre **Deployment** + **ReplicaSet**.

Deployment come Livello Superiore

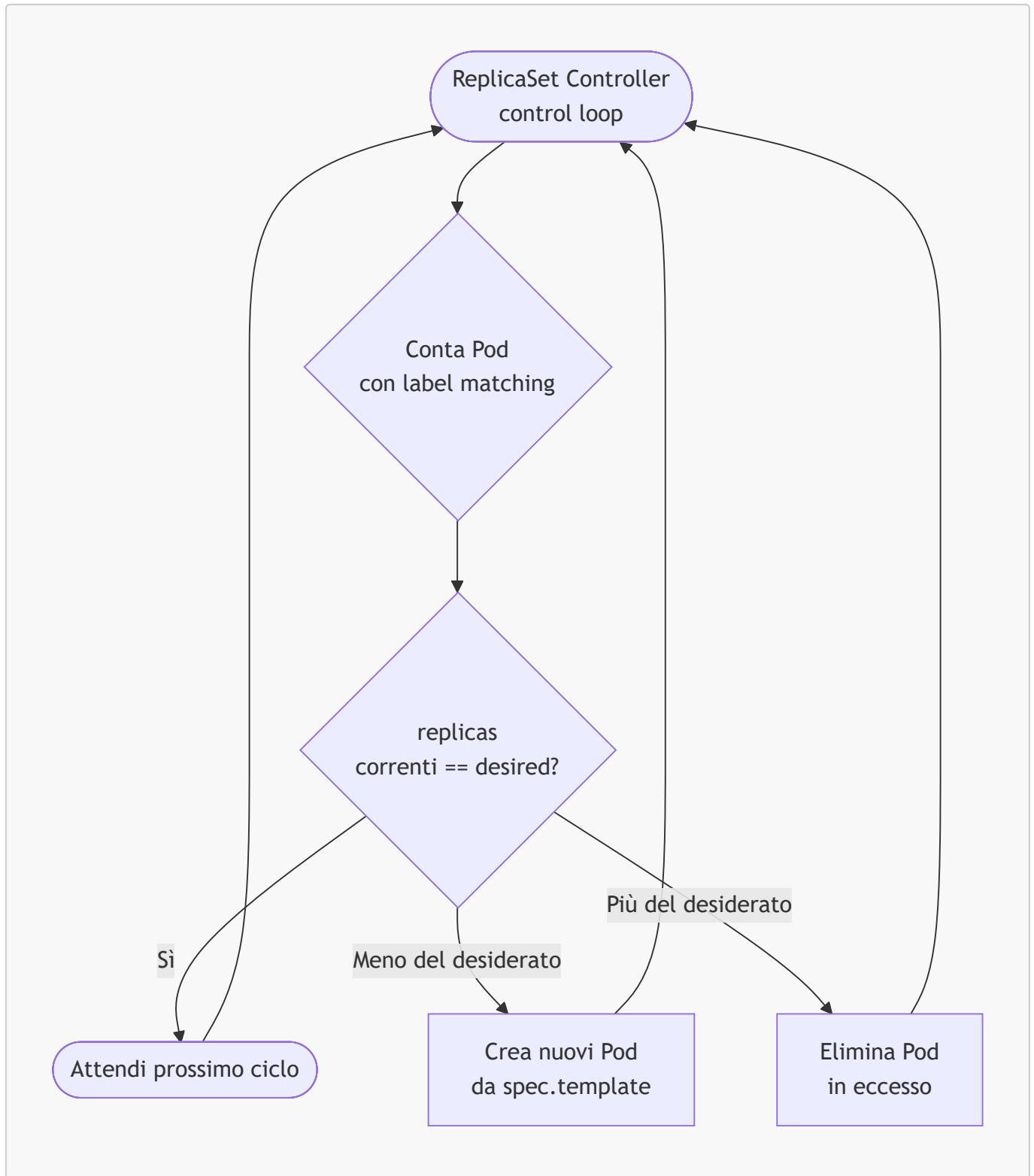
In produzione non si crea mai un **ReplicaSet** direttamente. Si usa invece un **Deployment**, che:

1. Crea e gestisce **ReplicaSet** automaticamente
2. Supporta rolling update (crea un nuovo RS, scala su il nuovo, scala giù il vecchio)
3. Mantiene la cronologia delle revisioni (oc rollout history)
4. Permette rollback (oc rollout undo)

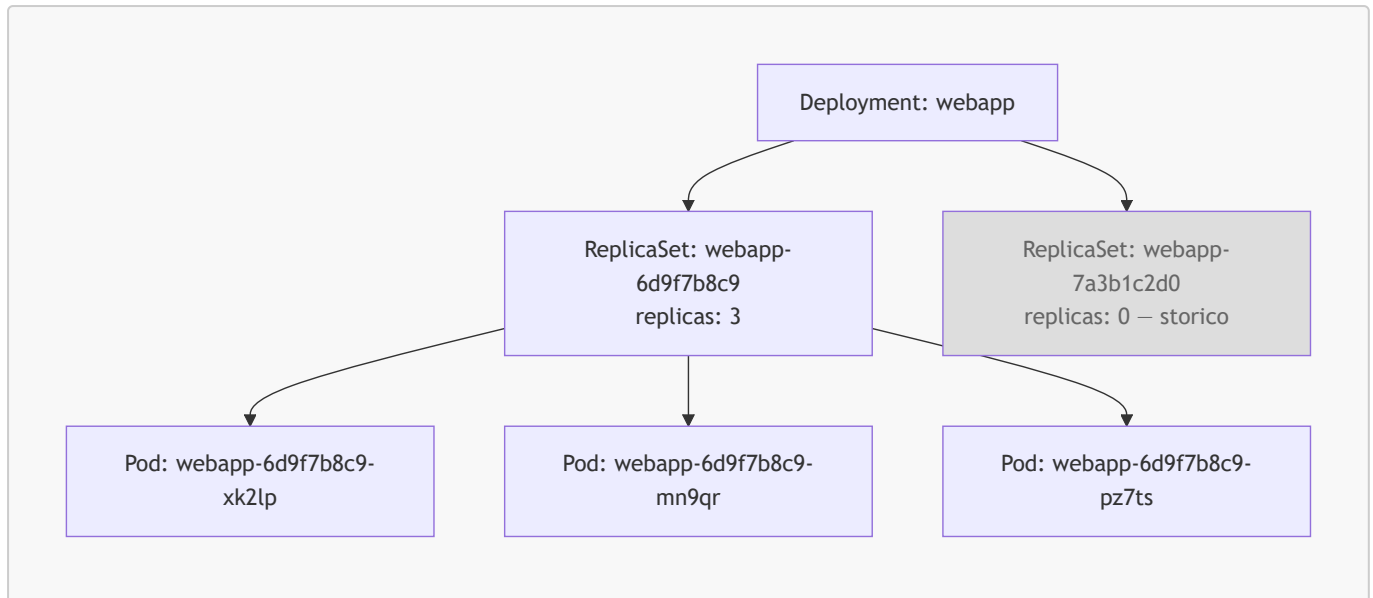
```
Deployment
├─ ReplicaSet (revisione corrente, es. webapp-6d9f7b8c9)
│   ├── Pod webapp-6d9f7b8c9-xk2lp
│   ├── Pod webapp-6d9f7b8c9-mn9qr
│   └─ Pod webapp-6d9f7b8c9-pz7ts
```

Diagrammi

Meccanismo di Riconciliazione del ReplicaSet



Gerarchia Deployment → ReplicaSet → Pod



Comandi Pratici con Output Atteso

```
# Elencare tutti i ReplicaSet nel progetto corrente
oc get replicaset
```

```
# Output atteso:
```

# NAME	DESIRED	CURRENT	READY	AGE
# webapp-6d9f7b8c9	3	3	3	25m
# webapp-7a3b1c2d0	0	0	0	2h

```
# Mostrare i dettagli di un ReplicaSet
oc describe replicaset webapp-6d9f7b8c9
```

```
# Output atteso (estratto):
```

```
# Name:          webapp-6d9f7b8c9
# Namespace:     myproject
# Selector:      app=webapp,tier=frontend
# Labels:        app=webapp
# Controlled By: Deployment/webapp
# Replicas:      3 current / 3 desired
# Pods Status:   3 Running / 0 Waiting / 0 Succeeded / 0 Failed
```

```
# Scalare direttamente un ReplicaSet (sconsigliato in produzione: usare il
Deployment)
```

```
oc scale replicaset webapp-6d9f7b8c9 --replicas=5
```

```
# Output atteso:
```

```
# replicaset.apps/webapp-6d9f7b8c9 scaled
```

```
# Verificare il risultato dello scaling
oc get pods -l app=webapp
```

```
# Output atteso:
```

# NAME	READY	STATUS	RESTARTS	AGE
# webapp-6d9f7b8c9-xk2lp	1/1	Running	0	26m
# webapp-6d9f7b8c9-mn9qr	1/1	Running	0	26m
# webapp-6d9f7b8c9-pz7ts	1/1	Running	0	26m
# webapp-6d9f7b8c9-ab4cd	1/1	Running	0	12s
# webapp-6d9f7b8c9-ef5gh	1/1	Running	0	12s

```
# Eliminare un Pod manualmente – il ReplicaSet ne creerà subito uno nuovo
oc delete pod webapp-6d9f7b8c9-xk2lp
```

```
# Monitorare la sostituzione in tempo reale
```

```
oc get pods -l app=webapp --watch
```

```
# Output atteso:
```

# webapp-6d9f7b8c9-xk2lp	1/1	Terminating	0	30m
# webapp-6d9f7b8c9-ij6kl	0/1	Pending	0	1s
# webapp-6d9f7b8c9-ij6kl	0/1	Running	0	5s
# webapp-6d9f7b8c9-ij6kl	1/1	Running	0	12s

```
# Visualizzare lo YAML di un ReplicaSet esistente
oc get replicaset webapp-6d9f7b8c9 -o yaml
```

```
# Mostrare i ReplicaSet con il numero di repliche e la relazione col Deployment
oc get replicaset -o wide
```

```
# Output atteso:
```

# NAME	DESIRED	CURRENT	READY	AGE	CONTAINERS	IMAGES
SELECTOR						
# webapp-6d9f7b8c9	3	3	3	30m	webapp	
.../ubi9/ubi-minimal:9.4		app=webapp				

```
# Elencare i ReplicationController (legacy) presenti nel namespace
oc get replicationcontrollers
```

```
# Output atteso (se presenti DeploymentConfig legacy):
```

# NAME	DESIRED	CURRENT	READY	AGE
# backend-1	2	2	2	3d

```
# Verificare il campo ownerReference di un Pod per capire chi lo gestisce
oc get pod webapp-6d9f7b8c9-mn9qr -o jsonpath='{.metadata.ownerReferences}' |
python3 -m json.tool

# Output atteso:
# [{"apiVersion": "apps/v1", "blockOwnerDeletion": true, "controller": true,
#   "kind": "ReplicaSet", "name": "webapp-6d9f7b8c9", "uid": "abc-123..."}]
```

Esempi YAML Completi

ReplicaSet con matchLabels

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: webapp-rs
  namespace: myproject
  labels:
    app: webapp
    managed-by: corso-ocp
spec:
  # Numero desiderato di Pod
  replicas: 3
  # Selettore per identificare i Pod gestiti da questo RS
  selector:
    matchLabels:
      app: webapp
      tier: frontend
  # Template del Pod da creare
  template:
    metadata:
      labels:
        # Devono corrispondere al selector sopra
        app: webapp
        tier: frontend
    spec:
      containers:
        - name: webapp
          image: registry.access.redhat.com/ubi9/ubi-minimal:9.4
          ports:
            - containerPort: 8080
          resources:
            requests:
              memory: "64Mi"
              cpu: "50m"
            limits:
              memory: "128Mi"
              cpu: "200m"
```

 **Suggerimento:** Creare questo ReplicaSet direttamente con `oc apply -f replicaset.yaml` è didatticamente utile per comprendere il meccanismo. In produzione, usa sempre un `Deployment`.

ReplicaSet con matchExpressions

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: webapp-rs-expr
  namespace: myproject
spec:
  replicas: 2
  selector:
    matchExpressions:
      # Seleziona Pod con app = webapp 0 webapp-v2
      - key: app
        operator: In
        values:
          - webapp
          - webapp-v2
      # Esclude Pod con tier = database
      - key: tier
        operator: NotIn
        values:
          - database
  template:
    metadata:
      labels:
        app: webapp
        tier: frontend
    spec:
      containers:
        - name: webapp
          image: registry.access.redhat.com/ubi9/ubi-minimal:9.4
          resources:
            requests:
              memory: "64Mi"
              cpu: "50m"
            limits:
              memory: "128Mi"
              cpu: "200m"
```

ReplicationController (Legacy — solo a scopo illustrativo)

```
# NOTA: ReplicationController è DEPRECATO. Mostrato solo per riconoscimento
legacy.
apiVersion: v1
kind: ReplicationController
metadata:
  name: backend-rc
```



```
namespace: myproject
spec:
  replicas: 2
  # Il selector del RC supporta solo uguaglianza semplice (no matchExpressions)
  selector:
    app: backend
  template:
    metadata:
      labels:
        app: backend
    spec:
      containers:
        - name: backend
          image: registry.access.redhat.com/ubi9/ubi-minimal:9.4
          ports:
            - containerPort: 8080
```

Note Specifiche per Azure ARO

DeploymentConfig e ReplicationController su ARO

Prima di OCP 4.14, OpenShift usava **DeploymentConfig** (DC) come oggetto nativo per i deploy, il quale creava **ReplicationController** invece di **ReplicaSet**. Su ARO con OCP 4.14+ i DC sono deprecati e Red Hat raccomanda la migrazione a **Deployment**:

```
# Verificare se esistono DeploymentConfig nel namespace
oc get deploymentconfigs -n myproject

# Migrare un DC a Deployment
oc get dc <nome> -o yaml > dc-backup.yaml
# Poi modificare manualmente o usare il tool oc-dc-to-deploy
```

Scaling e ARO Cluster Autoscaler

Su ARO è possibile abilitare il **Cluster Autoscaler**: se un ReplicaSet richiede più nodi di quanti disponibili, l'autoscaler richiede ad Azure di aggiungere nodi al MachineSet. Per verificare:

```
# Verificare se il cluster autoscaler è attivo
oc get clusterautoscaler

# Verificare il MachineAutoscaler per i worker node
oc get machineautoscaler -n openshift-machine-api
```

 **Suggerimento:** Su ARO, imposta sempre **resources.requests** precisi nei Pod: il Cluster Autoscaler usa i **requests** (non i **limits**) per decidere se aggiungere nodi.

Riepilogo dei Punti Chiave

- Il **ReplicaSet** garantisce che un numero preciso di Pod con label specificati siano sempre in esecuzione tramite il reconciliation loop
 - **matchLabels** è la forma semplice del selettore; **matchExpressions** supporta operatori **In**, **NotIn**, **Exists**, **DoesNotExist**
 - Il RS adotta Pod esistenti con label corrispondenti: attenzione a non creare Pod "orfani" con label che collidono con un RS attivo
 - Il **ReplicationController** è il predecessore deprecato del **ReplicaSet**, con selettori meno potenti
 - In produzione non si crea mai un RS direttamente: si usa un **Deployment** che gestisce RS con supporto a rolling update e rollback
 - Eliminare un Pod manualmente non riduce il numero di repliche: il RS ne crea subito uno sostitutivo
 - Su ARO, i **DeploymentConfig** legacy creano **ReplicationController**; si raccomanda la migrazione a **Deployment**
-

Domande di Verifica

1. Qual è la principale differenza tra **matchLabels** e **matchExpressions** nel selettore di un **ReplicaSet**?

- A) **matchLabels** è più recente e supporta più operatori
- B) **matchExpressions** supporta operatori come **In**, **NotIn**, **Exists** che **matchLabels** non supporta ✓
- C) Non c'è differenza pratica
- D) **matchExpressions** può selezionare solo Pod nello stesso namespace

2. Perché in produzione non si crea direttamente un **ReplicaSet**?

- A) I **ReplicaSet** non sono supportati su OCP 4.14+
- B) Il **ReplicaSet** non supporta più di 10 repliche
- C) Un **Deployment** gestisce i **ReplicaSet** e aggiunge supporto a rolling update e rollback ✓
- D) I **ReplicaSet** non possono essere scalati

3. Cosa succede se si elimina manualmente un Pod gestito da un **ReplicaSet** con **replicas: 3**?

- A) Il **ReplicaSet** scala a 2 repliche permanentemente
- B) Il **ReplicaSet** crea immediatamente un nuovo Pod per tornare a 3 repliche ✓
- C) Il **ReplicaSet** si blocca finché non viene riavviato
- D) Il Pod viene ricreato solo al prossimo deploy

4. Cosa distingue un **ReplicationController** da un **ReplicaSet**?

- A) Il **ReplicationController** supporta **matchExpressions**; il **ReplicaSet** no
- B) Il **ReplicaSet** è deprecato; il **ReplicationController** è il componente corrente
- C) Il **ReplicationController** ha selettori solo per uguaglianza semplice ed è deprecato; il **ReplicaSet** ha selettori avanzati ✓
- D) Non c'è differenza funzionale tra i due

5. Un **ReplicaSet** trova 4 Pod con le label corrette ma **spec.replicas** è impostato a 3. Cosa fa il controller?

- A) Aumenta automaticamente `spec.replicas` a 4
- B) Ignora il Pod in eccesso
- C) Elimina uno dei 4 Pod per tornare al numero desiderato 
- D) Genera un errore e si blocca